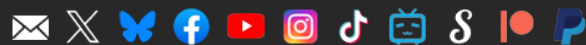


Inigo Quilez :: articles :: float, small and random - 2005



Here's a little trick I invented back in 2005 while creating the [195/95/256 64 kilobyte demo](#), when I learnt to never underestimate how slow a "int 2 float / float 2 int" data conversion can be. The problem came when debugging the software synthesizer made by Marc (aka Gortu). After several tests and profile sessions, we found something really astonishing: one of the bottlenecks of the synth was the noise generator. Our noise generator was basically creating an integer pseudo-random number, then casting it to float and finally scaling it down to the $[-1, 1]$ range. We were using the same congruent generator as Visual Studio's implementation in their stdlib, and then we were simply converting to floating point and normalizing the result to the $[-1, 1]$ range. Here goes the original code:

```
float sfrand( int *seed )
{
    *seed = 0x00269ec3 + (*seed)*0x000343fd;
    int a = ((*seed)>>16) & 32767;
    return -1.0f + (2.0f/32767.0f)*((float)a);
}
```

Before diving in, note how the function takes a seed as parameter. As you probably know the regular Ansi C `rand()` function doesn't take any argument, which makes it impossible to be used in a multithreaded environment, like a raytracer or a demo like ours where both sound synthesizer and renderer run at the same time and need to generate random numbers. So, I usually pass the seed as a parameter. You can think of it as a "context" for the function: you should keep one for each thread, probably on the stack. Anyway.

Now back to generating random floating point numbers; it seems like the code above is quite simple and should run fast, right? There's no divisions, just a few integer operations, the conversion, and an `fma` instruction to scale and bias the result. However, as I said, the cast from int to float was simply killing performance. It's quite well known that the `fld` instruction is slow (this is a x86 FPU instruction, these days CPUs use XMM registers and SSE instruction to do floating point math, but the principle still applies). But we never expected it to be a bottleneck for the whole demo, so I had to find a way to create random floating point numbers much quicker.

The idea was really simple, and worked quite well: generate random bits for the mantissa of our floating point number. But setting the appropriate bits in the exponent of my float (see [the IEEE 754 standard](#)) I could ensure the random mantissa bits would make a number always in the range $[-1, 1]$ or $[0, 1]$ as needed. Have a

look below to the layout of the bits on the floating point format:

```

33      22      0
10      32      0
seee eeee efff ffff ffff ffff ffff ffff

```

where

s = sign bit

e = exponent

f = fractional part of the mantissa

value = $s * 2^{(e-127)} * m$, where $m = 1.f$, and thus $1 \leq m < 2$

In particular, because of the way the IEEE 754 standard is defined, the random mantissa will generate a number between 1 and 2, if we fix our exponent to 127 and the sign bit to zero. Then we can afterwards scale (by 2.0) and offset it (by -3.0) to make it fit in the segment $[-1, 1)$.

However, I realized this can be done a bit better and avoid the scaling if we directly generate a float random number between 2 and 4. For that to be happen, the only thing to do is to force the exponent to be 128, so that the output value is

value = $s * 2 * m$

That belongs to the range $[2, 4)$. So, first operation to do to the 32 random bits is to mask the sign and exponent bits with

```

seee eeee efff ffff ffff ffff ffff ffff
0000 0000 0111 1111 1111 1111 1111 1111

```

or 0x007fffff in hexadecimal. Then just the exponent is set to 128 (10000000 in binary), with the next bit pattern

```

seee eeee efff ffff ffff ffff ffff ffff
0100 0000 0000 0000 0000 0000 0000 0000

```

or 0x40000000 in hexadecimal. So, finally, the complete signed floating point random generator looks like:

```

float sfrand( int *seed )
{

```

```
float res;  
*seed = 0x00269ec3 + (*seed)*0x000343fd;  
*((unsigned int *) &res) = (((unsigned int)(*seed))>>9) | 0x40000000;  
return( res-3.0f );  
}
```

Some compilers might complain about creating a pointer to an int data that points to a location reserved for a float. If that's the case, you can workaround it by using a union (thx Reinder Nijhoff for the suggestion). The code for a function that returns a random number between 0 and 1 would then look like this:

```
float frand( int *seed )  
{  
    union  
    {  
        float fres;  
        unsigned int ires;  
    };  
  
    *seed = 0x00269ec3 + (*seed)*0x000343fd;  
    ires = (((unsigned int)*seed)>>9) | 0x3f800000;  
    return fres - 1.0f;  
}
```

These two versions cannot be simpler and faster, perfect for a 4k or 64k intro! One can find congruent generators that do not have a bias term and simply do a 32 bit multiplication, but at that point the wins are much smaller. When I made some measures to test the performance of this method, I got 4 times the performance of my original implementation. Regarding the quality, this generator beats a normal integer random value with rand() and then scaling it as the original function shown here does. Remember this one generates 23 random bits instead of 15. So, what else can we ask for our improved random number generator?

inigo quilez - learning computer graphics since 1994